# STUDENT MANAGEMENT REST API

Spring Boot Assignment Report

SUBMITTED TO: DEEP PRAKASH SIR

SUBJECT: PROJECT BASED LEARINING IN JAVA

SUBMITTED BY:

Harsh Raj– 24BCS12444

Section -24BCS-KRG-617A

# 1. Project folder structure

```
student-management-api/
├── src/
│   ├── main/
│   │   ├── java/com/example/demo/
│   │   │   ├── DemoApplication.java
│   │   │   ├── controller/
│   │   │   │   └── StudentController.java
│   │   │   ├── service/
│   │   │   │   ├── StudentService.java
│   │   │   │   └── StudentServiceImp.java
│   │   │   ├── repository/
│   │   │   │   └── StudentRepository.java
│   │   │   └── entity/
│   │   │       └── Student.java
│   │   └── resources/
│   │       └── application.properties
│   └── test/
├── screenshots/
├── README.md
├── .gitignore
└── pom.xml
```

# 2. Database configuration

The project is intended to use PostgreSQL. Create a database before starting the application. Configure the connection in src/main/resources/application.properties. Replace the placeholders with the local database name, username, and password. Do not commit real passwords or secrets to GitHub.

```
spring.datasource.url=jdbc:postgresql://localhost:5432/Student
spring.datasource.username=postgres
spring.datasource.password=251995
spring.datasource.driver-class-name=org.postgresql.Driver
spring.jpa.hibernate.ddl-auto=update
spring.jpa.show-sql=true
spring.jpa.database-platform=org.hibernate.dialect.PostgreSQLDialect
```

**Configuration notes**: ensure the PostgreSQL JDBC driver dependency is present in pom.xml. The database named student_db must exist unless the database has been created separately. The value update asks Hibernate to update the schema to reflect entity changes; for production systems, use a controlled database migration strategy instead.

| Attribute | Description |
|---|---|
| id | Primary Key |
| name | Student name |
| email | Student email address |
| course | Enrolled course |
| age | Student age |

### Example database creation

```
CREATE DATABASE student_db;
```

Set DB_USERNAME and DB_PASSWORD in the local run environment, or use a local-only configuration that is excluded from version control. Never publish credentials in the report or repository.

# 3. REST API endpoint specification

The following endpoint paths follow the API specification shared for this project. The base URL used in the examples is http://localhost:8081.

| HTTP Method | Endpoint | Description |
|---|---|---|
| POST | `/api/students` | Add a new student |
| GET | `/api/students` | Retrieve all students |
| GET | `/api/students/{id}` | Retrieve student by ID |
| PUT | `/api/students/{id}` | Update student details |
| DELETE | `/api/students/{id}` | Delete a student |
| GET | `/api/students/course/{course}` | Retrieve students by course |

### Example request body for POST and PUT

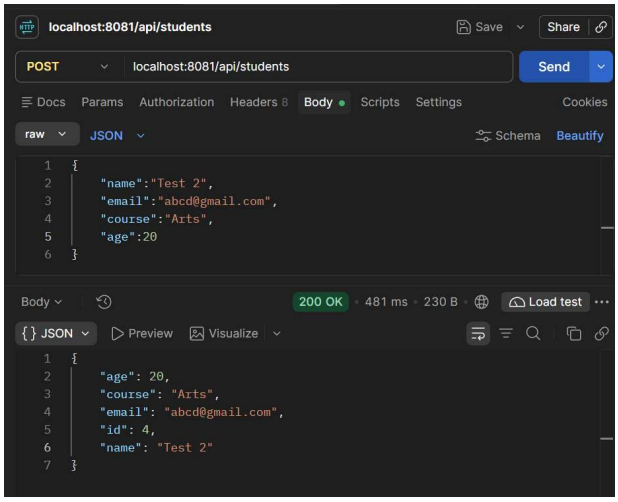
```
{
  "name": "Sample Student",
  "email": "student@example.com",
  "course": "Java",
  "age": 21
}
```

For POST, the database should generate the ID if the Student entity uses an appropriate @GeneratedValue strategy. For PUT, the request body contains the replacement values supported by the implemented update method.

# 4. API testing evidence

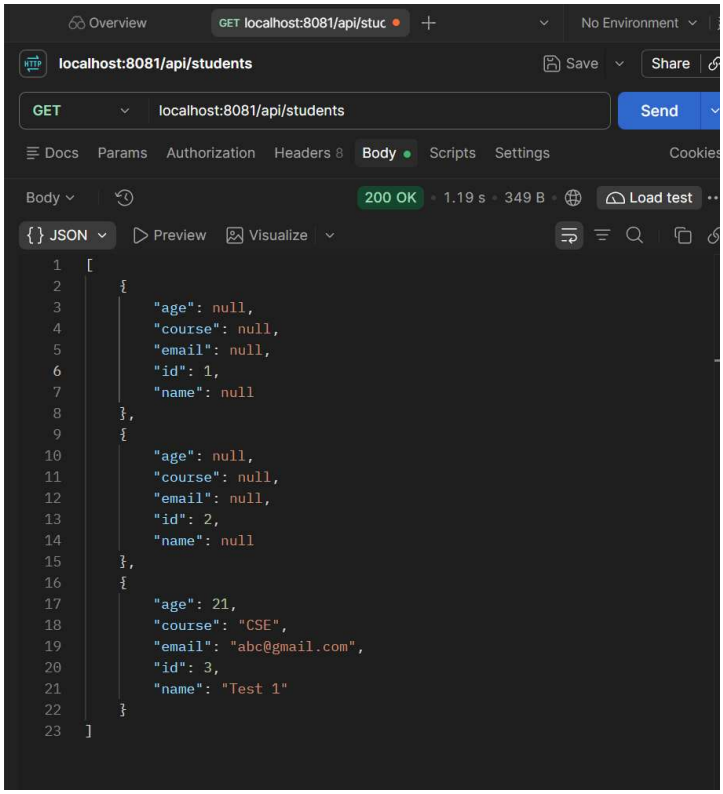
## Screenshot 1 – Add student

**Request:** POST /api/students



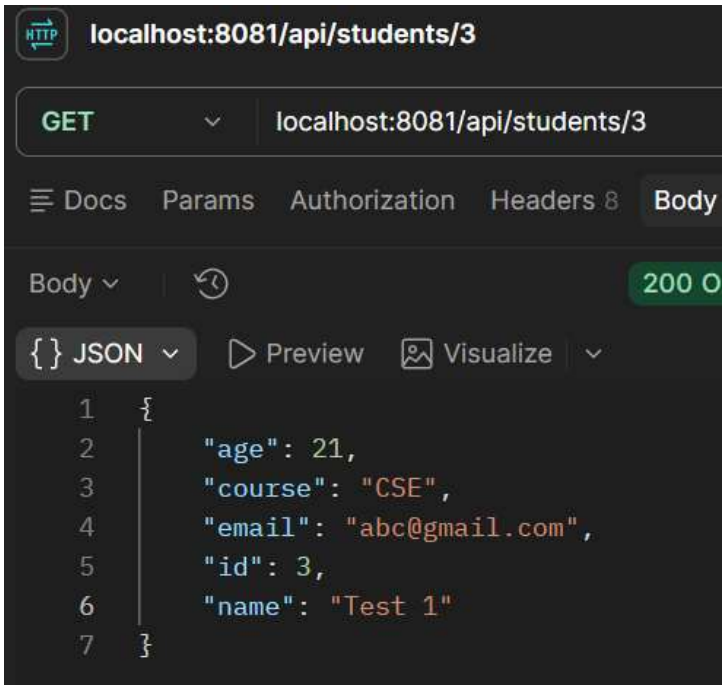

## Screenshot 2 – Get all students

**Request:** GET /api/students



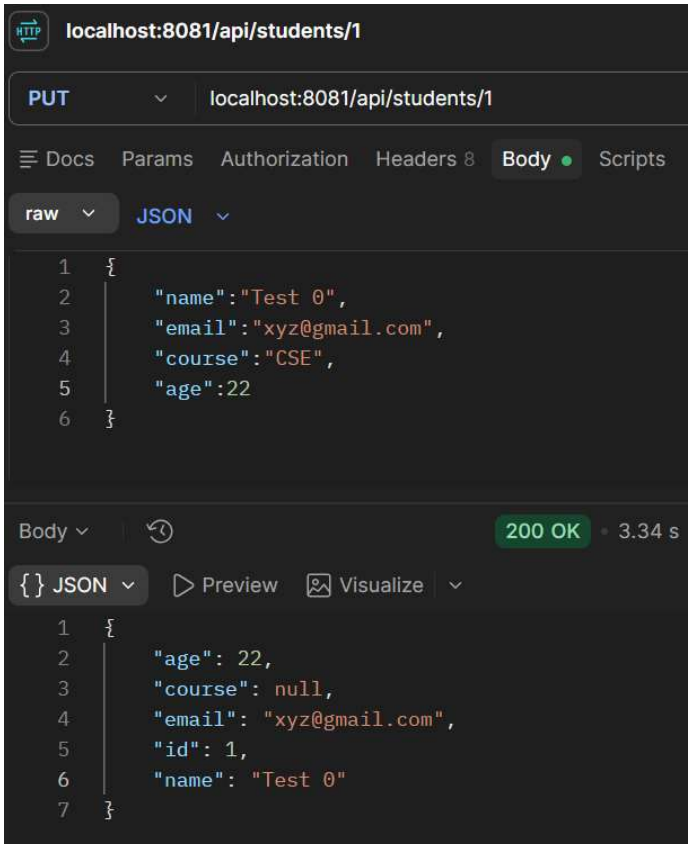
## Screenshot 3 – Get student by ID

**Request:** GET /api/students/1



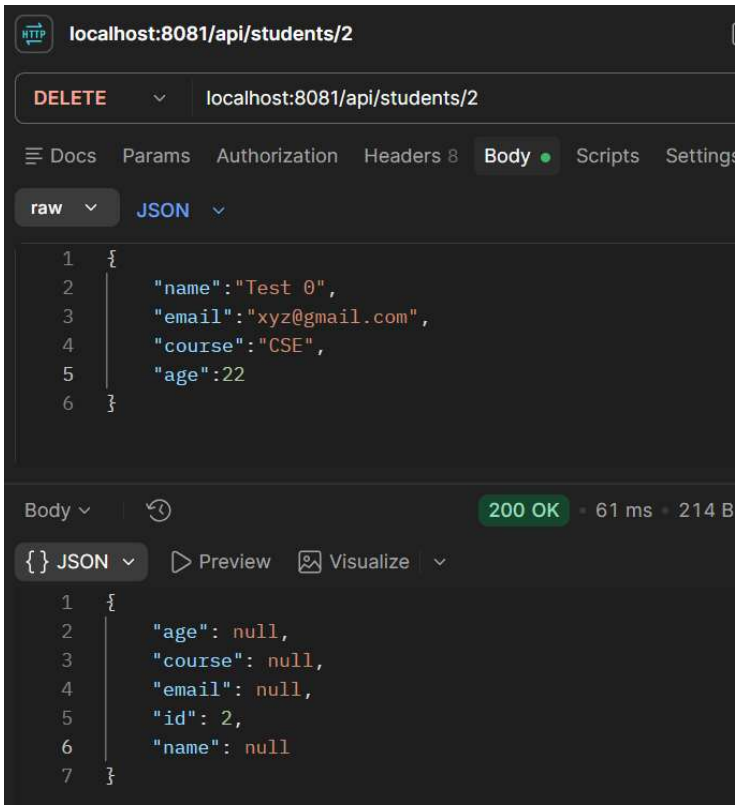

## Screenshot 4 – Update student

**Request:** PUT /api/students/1



## Screenshot 5 – Delete student

**Request:** DELETE /api/students/1




## Screenshot 6 – Filter by course

**Request:** GET /api/students/course/Java

